DBMGNet: A Dual-Branch Mamba-GCN Network for Hyperspectral Image Classification

Hao Wang, Peixian Zhuang, *Senior Member, IEEE*, Xiaochen Zhang, and Jiangyun Li

***Abstract*— In hyperspectral image (HSI) classification, convolutional neural networks (CNNs) excel at local feature modeling but are limited to Euclidean space. Transformers offer long-range dependence modeling but suffer from high computational complexity. In contrast, graph convolutional networks (GCNs) can process information in non-Euclidean space, compensating for the limitations of CNNs. Meanwhile, the state-space model (SSM) Mamba, thanks to its linear complexity and strong long-range dependence modeling, shows great potential to offer an alternative to Transformers for HSI classification. To address the limitations of CNNs and Transformers while exploiting the potential of Mamba, we propose a dual-branch hybrid architecture named DBMGNet that combines Mamba with GCN for the HSI classification. In the Mamba branch, we design band selection enhanced bidirectional Mamba (BSEBM), which leverages Mamba's long-range dependence modeling and sequential modeling capabilities to process spatial–spectral information. In the GCN branch, we apply reparameterized Chebyshev graph convolution (RCGC) to model similarity dependencies in non-Euclidean space, along with designing an adjacency matrix based on the intrinsic characteristics of HSIs. Extensive experiments demonstrate that our DBMGNet achieves the state-of-the-art performance of HSI classification against 13 mainstream approaches. The code for this work will be available at: https://github.com/Wanghao00pro/DBMGNet**

***Index Terms*— Dual-branch, graph convolutional network (GCN), hyperspectral image (HSI) classification, Mamba.**

I. Introduction

HYPERSPECTRAL imaging has emerged as a powerful tool for capturing detailed spectral signatures across diverse applications, ranging from remote sensing to environmental monitoring [1], [2], [3]. Among them, hyperspectral image (HSI) classification is an important technique [4] and has significantly evolved from relying on traditional pixel-based approaches to leveraging advanced deep learning techniques. This transition has facilitated the extraction and utilization of rich spectral information, improving the analysis of terrestrial features and broadening the applications of HSI across various domains. Initially, pixel-based classifiers, such as support vector machines (SVMs) [5] and decision trees [6], were extensively used to focus on spectral characteristics at an individual pixel level. However, these methods typically struggled with the high dimensionality [7] and interclass spectral similarity inherent in hyperspectral data.

Subsequently, the development of deep learning has profoundly promoted the HSI classification. Convolutional neural networks (CNNs) have exhibited their impressive ability to extract both spectral and spatial features for enhancing classification accuracy. For instance, Hu et al. [8] proposed a 1D-CNN to capture the spectral features of HSIs. Song et al. [9] presented a deep feature fusion network (DFFN), which utilized 2D-CNNs to extract the spatial features of local cubes and address gradient vanishing issues by incorporating residual learning and hierarchical feature fusion. The development of 3D-CNNs [10] represented another trend, processing spatial and spectral dimensions simultaneously to maintain the natural structure of hyperspectral data. These approaches allow for a deeper integration of spectral and spatial information, thus facilitating more precise classifications. Additionally, Yu et al. [11] proposed the spatial mapping expansion with spectral compression (SMESC) network for HSI classification, which introduced spatial mapping expansion and spectral compression to address subpixel feature loss in CNN. Although CNNs can extract local spatial–spectral relationships, they are difficult to effectively establish similarity relationships between pixels, due to the limitation of the grid sampling.

Unlike standard CNNs that process the data in a regular grid, graph convolutional networks (GCNs) [12] represent a burgeoning network architecture that processes graph-structured data by modeling the relationships between samples. This makes GCNs particularly suitable for HSIs, where spatial and spectral data points have complex interdependencies [13]. Li et al. [14] described the graph convolutional operation in GCN as a form of Laplacian smoothing, which is crucial for capturing pixel similarities. Chen et al. [15] proposed a novel graph network, called SS-GCN, which integrated both spatiotemporal and spectral-domain information using a two-stream graph topology. Recent studies show that GCN-based models excel in HSI classification.

Received 17 December 2024; revised 22 February 2025 and 21 March 2025; accepted 20 April 2025. Date of publication 25 April 2025; date of current version 27 May 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62171252 and in part by the Fundamental Research Funds for the Central Universities under Grant 00007764. *(Hao Wang and Peixian Zhuang contributed equally to this work.) (Corresponding author: Jiangyun Li.)*

Hao Wang and Xiaochen Zhang are with the School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing 100083, China (e-mail: wanghao00pro@163.com; zhangxc0105@163.com).

Peixian Zhuang is with the Key Laboratory of Knowledge Automation for Industrial Processes, Ministry of Education, and the School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing 100083, China (e-mail: zhuangpeixian0624@163.com).

Jiangyun Li is with the Key Laboratory of Knowledge Automation for Industrial Processes, Ministry of Education, the School of Automation and Electrical Engineering, and the Shunde Graduate School, University of Science and Technology Beijing, Beijing 100083, China (e-mail: leejy@ustb.edu.cn).

Digital Object Identifier 10.1109/TGRS.2025.3564364

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

Hong et al. [16] applied GCNs to represent and analyze irregular data in HSIs, significantly reducing the computational cost associated with traditional GCNs in large-scale remote sensing datasets. Liu et al. [17] designed a fast dynamic GCN and CNN (FDGC) parallel network for HSI classification, efficiently extracting spatial, spectral, and structural features while reducing time complexity and memory usage. Jiang et al. [18] proposed a graph generative structure-aware model (GraphGST) that introduces a new absolute positional encoding and self-supervised cross-view contrastive learning to enhance HSI classification. Bai et al. [19] introduced a graph attention network with an adaptive graph structure mining (GAT-AGSM) approach that leverages superpixel segmentation and a spatial–spectral attention layer to enhance HSI classification. Su et al. [20] proposed ACGT-Net, which uses a meta-heuristic cuckoo search strategy to adaptively refine graph structures, enhancing the generalization of graph neural network (GNN). Ding et al. [21] presented a diversity-connected GCN to improve graph structure quality through topological extensions and adaptive smoothing. Yu et al. [22] proposed a discriminant superpixel graph to enhance semi-supervised HSI classification by improving graph discriminability.

Recently, Transformers have demonstrated notable efficacy in HSI classification, due to their strong long-range modeling capabilities. By leveraging self-attention mechanisms, Transformers can dynamically focus on various segments of a hyperspectral cube. Hong et al. [23] introduced a novel backbone network named SpectralFormer (SF), which reconceptualized HSI classification using a sequential approach with transformers. Sun et al. [24] designed a spectral–spatial feature tokenization transformer (SSFTT) to capture high-level semantic information and spectral–spatial features, demonstrating strong performance on multiple HSI datasets. Roy et al. [25] introduced a spectral–spatial morphological attention transformer (Morphformer), which combined morphological convolution with an attention mechanism to enhance information interaction between tokens. Although the Transformer has powerful representation capabilities, its efficacy is heavily constrained by the sequence length. That is, long sequences lead to high computational complexity, while short ones limit the representation effectiveness.

To address above inherent limitation of the Transformer architecture, state-space models (SSMs) have emerged. SSMs derived from classical control theory have recently been recognized for their robust performance in sequence data processing. Notably, structured SSMs, introduced by Gu et al. [26], mark a significant advancement through achieving long-range sequence modeling with linear computational complexity. These models have been extended by subsequent research, such as Mamba [27], which introduced a selective mechanism to enhance context compression capabilities, and outperformed traditional Transformers in several benchmark tests. The application of SSMs has extended beyond traditional sequence data, finding the utility in computer vision tasks. Innovations such as vision Mamba (Vim) [28] and visual SSM (Vmamba) [29] have been developed to model long-range dependencies within visual data. These

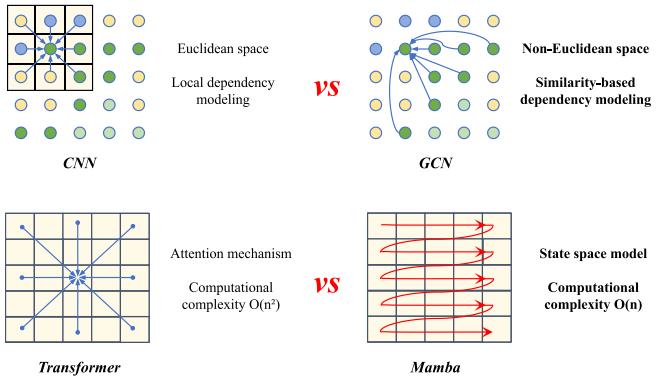


Fig. 1. Characteristics of four deep learning architectures. CNN only processes information within Euclidean space, while GCN handles information in non-Euclidean spaces and effectively establishes similarity dependencies. Transformer has quadratic computational complexity, whereas Mamba has linear computational complexity, and its sequential modeling capability is well-suited for continuous spectral information.

models have shown substantial efficacy across various domains including medical image analysis [30], video understanding [31], and other vision tasks [32], [33], but they primarily focus on RGB images. In the field of HSI classification, several Mamba-based methods have also been proposed. Huang et al. [34] presented the spectral–spectral Mamba (SSMamba) architecture, which leverages spectral–spatial token generation and stacked Mamba blocks for effective feature fusion. Zhou et al. [35] proposed Mamba-in-Mamba (MiM), which employs a centralized Mamba-cross-scan mechanism to convert HSI data into sequences and enhances feature extraction through a tokenized Mamba encoder with multiscale fusion. However, the application of SSMs in HSI classification has not been fully explored. In addition to their lower complexity, SSMs with sequence modeling capabilities are better suited to the spectral continuity characteristics of HSIs compared to Transformers, which is usually overlooked by existing Mamba-based HSI classification methods. This highlights a significant opportunity for leveraging SSMs to handle complex spectral characteristic of HSIs.

As previously mentioned, a single network architecture has inherent characteristics and limitations. Consequently, numerous hybrid architectures have been developed. Common examples include dual-branch hybrid models based on CNN and Transformer architectures, such as convolution transformer mixer (CTMixer) [36] and DBCTNet [37]; hybrid architecture [38] combining CNN with GCN; and hybrid model [39] integrating Transformer and graph structure. These hybrid architectures combine the strengths of different frameworks, achieving superior model performance. However, hybrid architectures based on Mamba have been extensively unexplored and remain an area for future research.

As illustrated in Fig. 1, the limitations of CNNs and Transformers can be, respectively, complemented by GCNs and Mamba.

1) CNN only processes information within Euclidean space due to grid sampling, while GCN handles information in non-Euclidean spaces and establishes similarity relationships between pixels.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

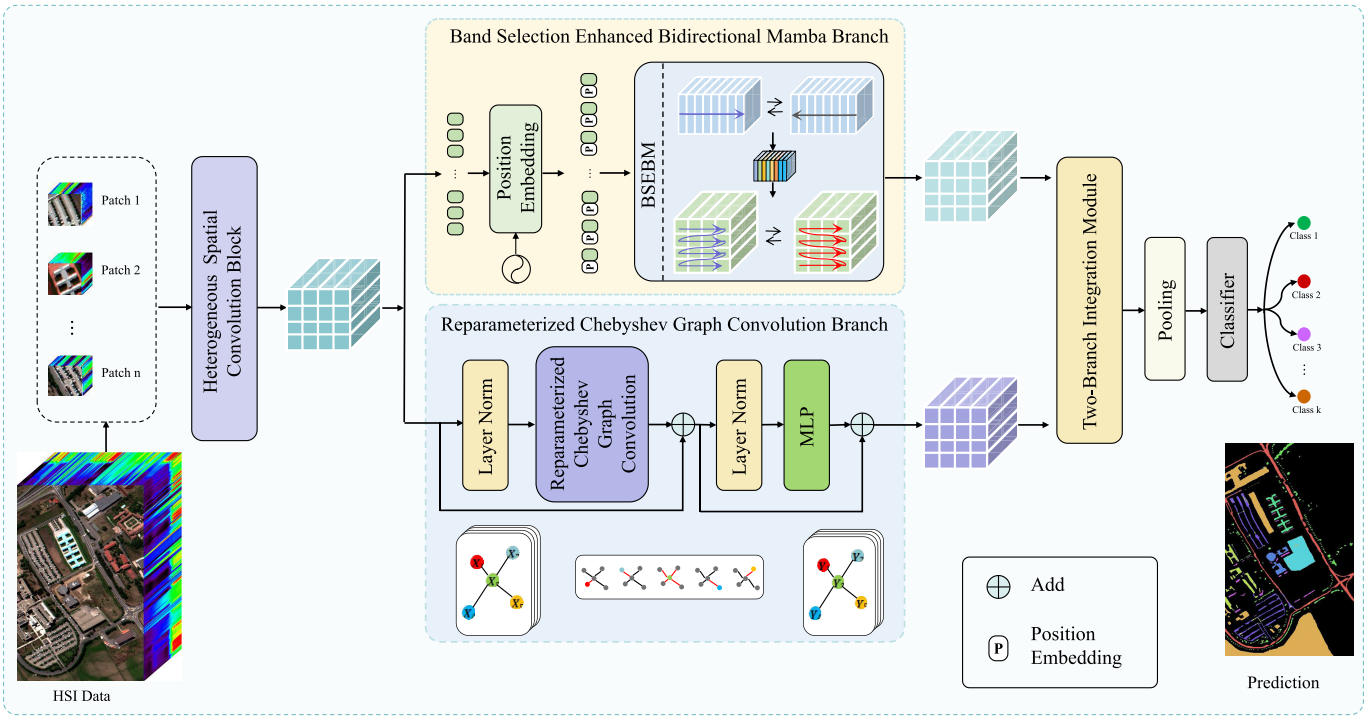


Fig. 2. Overall architecture of DBMGNet. Raw HSIs are divided into patches and processed through the heterogeneous spatial convolution block for feature extraction. The extracted features are then separately handled by the BSEBM branch and the RCGC branch. Subsequently, the TIM effectively fuses the outputs of the two branches, and these fused features are passed through pooling and a classifier to obtain final classification results.

2) The quadratic complexity of Transformer limits its computational efficiency, especially as sequence length increases. In contrast, Mamba has linear complexity to effectively address this limitation; meanwhile, its sequential modeling capabilities make it well-suited for processing continuous spectral information.

To address these limitations, this study proposes a novel hybrid architecture based on a dual-branch structure that combines Mamba and graph convolution tailored for HSI classification. Our model processes the hyperspectral data using two distinct processing branches. The first branch undergoes a band selection enhanced bidirectional Mamba (BSEBM), aimed at enhancing the selectivity of band information and establishing long-range dependencies of pixels. Meanwhile, the second branch is processed via a reparameterized Chebyshev graph convolution (RCGC), designed to model the similarity between pixels. The outputs of the dual branches are fused through a two-branch integration module (TIM), synergistically leveraging the information from both branches. This architecture is completed with a classification head that determines final class labels. The contributions of this work are summarized as follows.

1) To the best of authors' knowledge, the proposed DBMGNet is the first dual-branch network combining Mamba and GCN for the HSI classification. DBMGNet leverages the strengths of both Mamba and GCN architectures to form an effective hybrid collaborative network, which integrates pixel similarity relationships with long-range dependencies to improve the performance of HSI classification.
2) We propose the BSEBM branch, where the band selection Mamba (BSM) leverages Mamba's sequential modeling capabilities to process spectral information, while the bidirectional spatial Mamba captures spatial information. BSEBM designs a unique fusion method to fully integrate both spectral and spatial information for comprehensive feature representation. In this study, the RCGC is applied to HSI classification for the first time, and an adjacency matrix is designed for it based on the intrinsic characteristics of HSIs.
3) Extensive experiments on four public datasets demonstrate that our DBMGNet achieves better classification performance than the other 13 approaches. This confirms the effectiveness of Mamba hybrid architecture in HSI classification, paving the way for future research on Mamba hybrid architectures.

II. Proposed Methodology

A. Overall Architecture

The structure of our proposed DBMGNet is illustrated in Fig. 2. Initially, the raw HSI is divided into patches, and these patches are processed through a heterogeneous spatial convolution block to capture spatial features with varying receptive fields. Following this, the architecture bifurcates into two distinct pathways: BSEBM and RCGC. BSEBM uses Mamba to extract spatial and spectral features of hyperspectral data, while RCGC captures spectral similarity relationships among pixels. The outputs from both branches are effectively fused using the TIM. The fused features are then processed through pooling and a classifier to obtain final classification results.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

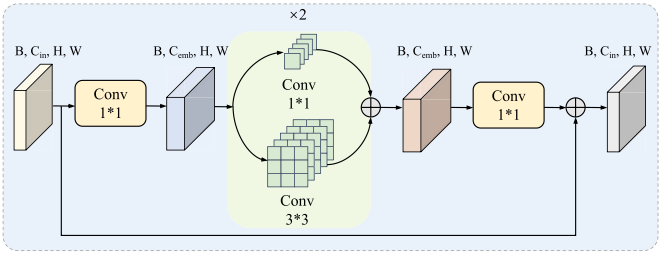


Fig. 3. Heterogeneous spatial convolution block. Heterogeneous convolutional kernels can extract spatial features with different receptive field sizes.

B. Heterogeneous Spatial Convolution Block

As shown in Fig. 3, the heterogeneous spatial convolution block is designed to capture information at varying spatial resolutions, enabling the model to better understand and represent complex patterns in the HSI data. First, it applies a convolution operation to map the spectral band dimension of the input $X \in \mathbb{R}^{B\times C_{\text{in}}\times H\times W}$ into a high-dimensional embedding space, where B is the batch size, C_{in} is the number of spectral bands, and H and W represent the patch size. Subsequently, it employs parallel convolutions with kernel sizes of 1×1 and 3×3 to extract spatial features from different receptive fields. Finally, a convolution operation is used to restore the embedding dimension back to the original input dimensionality of X. This architecture facilitates to capture both coarse-grained and fine-grained features, thereby enhancing the extraction of richer spatial information. The output of heterogeneous spatial convolution block can be expressed by

$$\begin{aligned} X_{\text{emb}} &= \text{Conv}_{1\times1}(X) \\ X_{\text{out}} &= \text{Conv}_{1\times1}(\text{Conv}_{1\times1}(X_{\text{emb}}) + \text{Conv}_{3\times3}(X_{\text{emb}})) \end{aligned} \tag{1}$$

where $X_{\text{emb}} \in \mathbb{R}^{B\times C_{\text{emb}}\times H\times W}$ represents the feature after mapping X to a high-dimensional embedding space. C_{emb} is set to 256. $X_{\text{out}} \in \mathbb{R}^{B\times C_{\text{in}}\times H\times W}$ is the final output of the heterogeneous spatial convolution block.

C. Band Selection Enhanced Bidirectional Mamba

The BSEBM branch of DBMGNet is designed to effectively classify HSIs by leveraging Mamba's ability to capture dependencies in both spectral and spatial dimensions. The core of the Mamba model is the use of SSMs. SSMs are fundamental statistical frameworks used to model the dynamic behavior of systems across various domains such as time-series analysis and control systems. These models represent a system's evolution as a continuous process, computed through a set of ordinary differential equations (ODEs). These ODEs project an input signal into a latent space and subsequently decode it into an output sequence, capturing the dynamic interactions within the system. The continuous form of an SSM can be expressed as follows:

$$\begin{aligned} h'(t) &= Ah(t) + Bx(t) \\ y(t) &= Ch(t) \end{aligned} \tag{2}$$

where $h(t) \in \mathbb{R}^N$ represents the latent state, $x(t) \in \mathbb{R}$ denotes the input signal, $y(t) \in \mathbb{R}$ is the output, $A \in \mathbb{R}^{N\times N}$ is the state transition matrix, and $B \in \mathbb{R}^N$ and $C \in \mathbb{R}^N$ are the projection matrices that map the input and latent state to the output, respectively. The latent state $h(t)$ encapsulates the system's internal states, which evolve over time according to the input $x(t)$ and the system's dynamics determined by A.

Due to the requirements of digital computation, the continuous SSM equations must be discretized for practical implementation. The Mamba module employs a zero-order hold (ZOH) technique to discretize these equations, transitioning from a continuous to a discrete-time model. This involves calculating the discrete equivalents of the state transition ($\tilde{A}$) and input matrices ($\tilde{B}$) as follows:

$$\begin{aligned} \tilde{A} &= \exp(\Delta A) \\ \tilde{B} &= (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B \end{aligned} \tag{3}$$

where Δ symbolizes the discretization step size. $\tilde{A}$ represents the discretized state transition matrix, while $\tilde{B}$ denotes the discretized input matrix. The variable I is the identity matrix. This discretization captures the system dynamics accurately over each time step, allowing for the state updates at discrete intervals. The discretized SSM can be expressed by

$$\begin{aligned} h_t &= \tilde{A}h_{t-1} + \tilde{B}x_t \\ y_t &= Ch_t \end{aligned} \tag{4}$$

where h_t indicates the latent state at time t. The output at time t is represented by y_t, and C is the output matrix. This discrete model ensures that the dynamics of the original continuous system are maintained while enabling the implementation on digital platforms.

In our proposed BSEBM branch, as shown in Fig. 4, we utilize a bidirectional spatial Mamba module to capture both forward and backward contextual spatial information. Additionally, we leverage the sequential modeling capabilities of Mamba to construct the BSM block, fully exploiting the continuous spectral band information of HSIs.

Specifically, the process begins with the application of position embedding to the input data, providing contextual information about the spatial arrangement of pixels. Afterward, a depthwise separable convolution (DSConv) is first applied to extract features. The data processed by the DSConv have three purposes. The forward mamba processes the data in the forward direction, extracting features using a series of linear transformations, convolutional layers, and activation functions [sigmoid linear unit (SiLU)]. This component captures the relationships in the sequence from the beginning to the end. Complementing the forward processing, the backward Mamba reverses the input sequence to model hidden features from the end to the beginning. This design allows our model to account for dependencies from both directions, enriching the representation of spatial information. This specific process is described as follows:

$$\begin{aligned} X_{\text{spaf}} &= \text{Mamba}_{\text{spaf}}(X_c) \\ X_{\text{spab}} &= \text{Flip}(\text{Mamba}_{\text{spab}}(\text{Flip}(X_c))) \end{aligned} \tag{5}$$

where X_c is the output of the DSConv, X_{spaf} is the output of the spatial forward Mamba, X_{spab} is the output of the spatial backward Mamba, and Flip$(\cdot)$ represents the sequence flipping

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

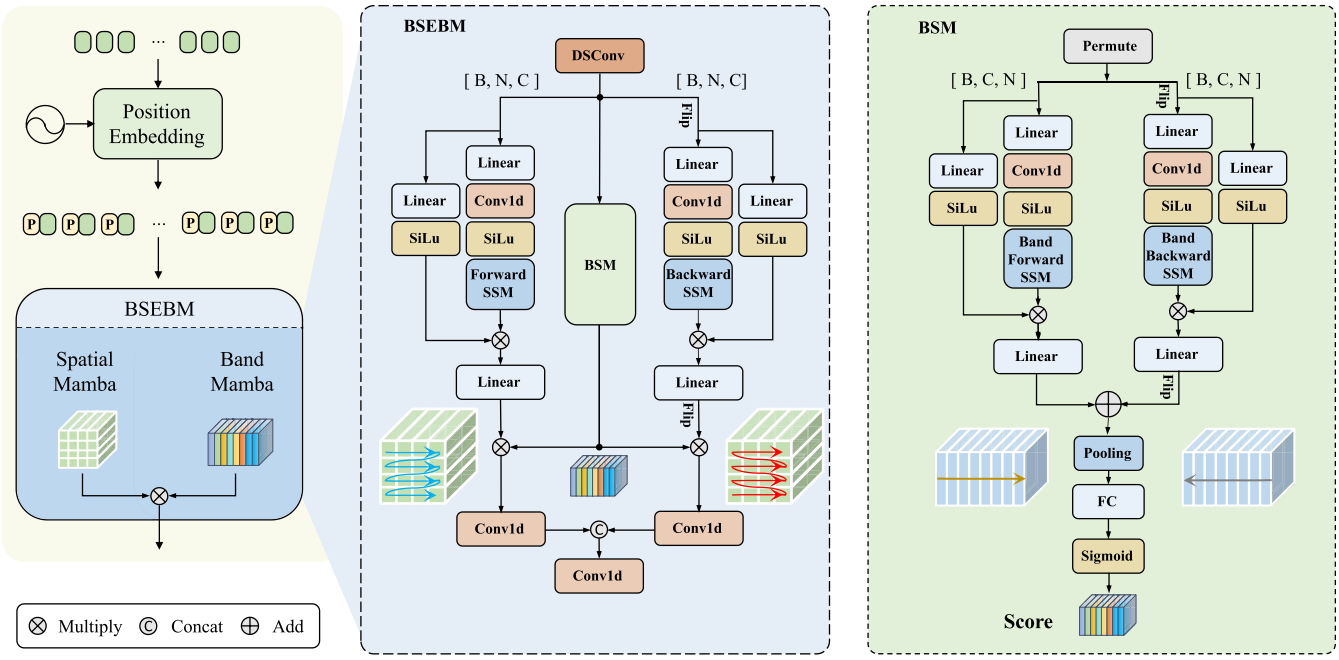


Fig. 4. BSEBM. On the spatial dimension, BSEBM leverages Mamba to perform bidirectional scanning to capture spatial information from HSI. Meanwhile, on the spectral dimension, BSM within BSEBM takes advantage of Mamba's sequential modeling capability to extract spectral scores to fully leverage the spectral information. Subsequently, the features from bidirectional Mamba are combined through 1-D convolutions.

operation. The Mamba$(\cdot)$ refers to the standard Mamba block introduced in [27].

Additionally, BSM exchanges spatial and spectral dimensions, and uses band forward Mamba and band backward Mamba to extract the spectral information of HSI. The bidirectional spectral information is then added. A channel attention mechanism is applied to the added spectral information, utilizing a squeeze-and-excitation block to generate a score representing the importance of different feature channels. This score is crucial to weight the spectral band of features, enabling our model to focus on the most relevant band information during classification. The process of obtaining the score is as follows:

$$\begin{aligned} X_r &= \text{Rearrange}(X_c) \\ X_b &= \text{Mamba}_{\text{spef}}(X_r) + \text{Flip}(\text{Mamba}_{\text{speb}}(\text{Flip}(X_r))) \\ S &= \text{Sigmoid}(\text{Linear}(\text{Pooling}(X_b))) \end{aligned} \tag{6}$$

where Rearrange$(\cdot)$ denotes the exchange of spatial and spectral dimensions of the features. X_b represents the combined results of the spectral forward Mamba and the spectral backward Mamba. S is the channel scores obtained from BSM.

The outputs from the bidirectional spatial Mamba are combined through an elementwise multiplication with the channel scores, enhancing the feature representation. The concatenated output from both pathways is passed through an additional convolution layer, producing a final integrated feature. This integrated feature serves as the final output of the BSEBM branch, which is expressed as follows:

$$\begin{aligned} Y_{\text{spaf}} &= X_{\text{spaf}} * S \\ Y_{\text{spab}} &= X_{\text{spab}} * S \\ Y_{\text{BSEBM}} &= \text{Conv}(\text{Concat}[\text{Conv}(Y_{\text{spaf}}), \text{Conv}(Y_{\text{spab}})]) \end{aligned} \tag{7}$$

where Y_{spaf} and Y_{spab} are the results of combining the spatial bidirectional Mamba with the channel scores, and Y_{BSEBM} is the final output of BSEBM.

D. Reparameterized Chebyshev Graph Convolution

As shown in Fig. 5, the RCGC allows our model to leverage the inherent graph-like structure of hyperspectral data, where each pixel is considered a node connected based on spectral similarity. The key of designing a graph convolution layer lies in the adjacency matrix and the feature propagation process. We design an adjacency matrix that is not a predefined fixed matrix; instead, it is derived from intrinsic features of HSIs. Moreover, in the feature propagation process, we employ reparameterized Chebyshev polynomials to efficiently propagate node features through the graph structure.

1) Adjacency Matrix: The design of the adjacency matrix is crucial in graph-based models, as it fundamentally defines the structure of the graph. An appropriately constructed adjacency matrix captures intrinsic relationships and connectivity among nodes, enabling effective feature propagation and learning. To this end, we construct an adjacency matrix based on spectral similarity relationships between hyperspectral pixels. First, we calculate the spectral similarity between pixel nodes using cosine similarity, resulting in a similarity matrix. The similarity matrix is defined as

$$S(i, j) = \frac{x_i \cdot x_j}{\|x_i\| \|x_j\|} \tag{8}$$

where $S(i, j)$ is the similarity score between pixel nodes i and j. x_i and x_j are the feature vectors corresponding to pixel nodes i and j. $\|x_i\|$ and $\|x_j\|$ are the Euclidean norms of the feature vectors. Then, we set a threshold T to filter the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

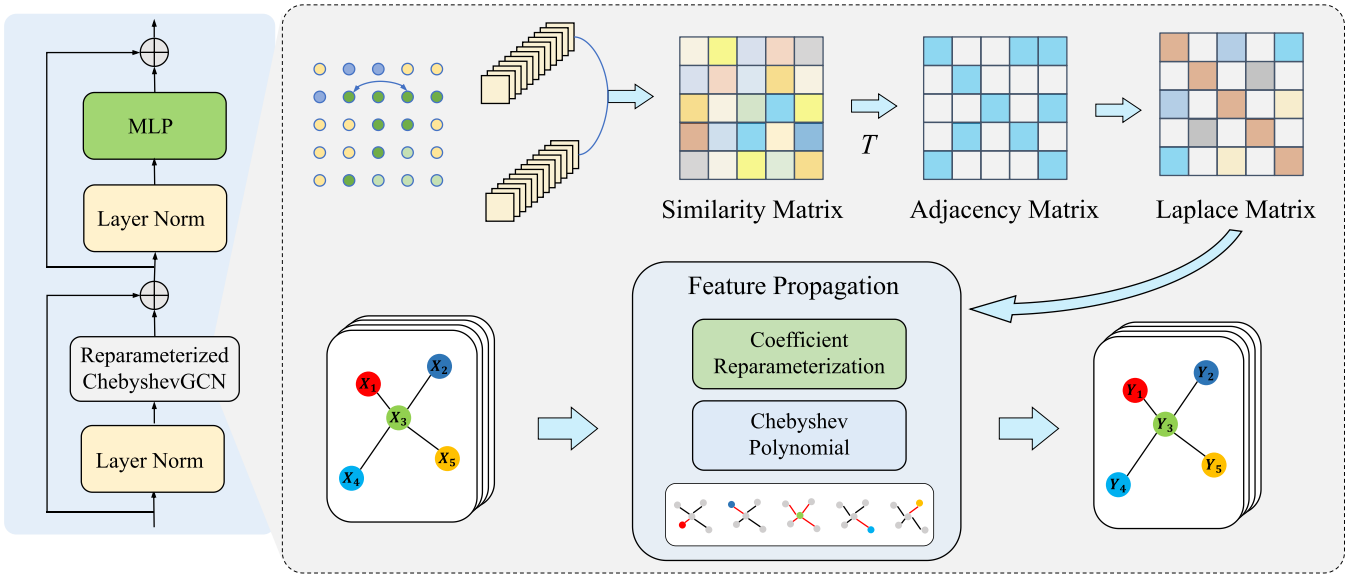


Fig. 5. In RCGC, we design the adjacency matrix based on intrinsic features of HSI and reparameterize the coefficients of the Chebyshev graph convolution. RCGC can establish pixel similarity dependencies in the non-Euclidean space, thereby extracting reliable classification features.

similarity matrix: values above T are assigned to be 1, while those below T are assigned as 0, yielding the final adjacency matrix, as shown in the following equation:

$$A(i,j) = \begin{cases} 1, & \text{if } S(i,j) > T \\ 0, & \text{if } S(i,j) \le T \end{cases} \tag{9}$$

where $A(i,j)$ is the final adjacency matrix. The benefit of this similarity filtering is that it enhances intraclass connections among nodes while reducing interclass connections, thereby improving the classification capability of the graph convolution and minimizing the interference effect from redundant information between classes.

2) Feature Propagation: In the feature propagation phase, we first compute the Laplacian matrix of the graph using the adjacency information. This provides us with a processed edge index and a normalization factor. The Laplacian matrix itself is not explicitly constructed; instead, we utilize edge indices and normalization factors to facilitate effective feature aggregation. Next, we add self-loops to the edge indices to ensure that each node retains its own features during the propagation process. Feature aggregation is then performed using updated edge indices and normalization factors, which are employed to combine features from neighboring nodes. The graph Laplacian L is defined as

$$L = I - D^{-1/2} A D^{-1/2} \tag{10}$$

where A is the adjacency matrix of the graph, D is the diagonal degree matrix with $D_{ii} = \sum_j A_{ij}$, and I is the identity matrix. This symmetric normalization of L facilitates the diffusion of features across the graph, emphasizing local pixel consistencies and discontinuities. The Laplacian matrix with self-loops added, denoted as $\tilde{L}$, is defined as follows:

$$\tilde{L} = L + I. \tag{11}$$

In spectral graph convolution, computing the eigen decomposition of the graph Laplacian matrix can be computationally expensive. Using the Chebyshev polynomial expansion [40] allows for an efficient approximation of the graph Laplacian operator. By representing the graph convolution as a series of Chebyshev polynomials, this method eliminates the need for direct matrix inversion or full adjacency matrix multiplication. This reduces computational complexity while retaining the ability to propagate and capture local neighborhood information. The Chebyshev iteration formula is presented as follows:

$$\begin{aligned} T_0(\tilde{L}) &= 1 \\ T_1(\tilde{L}) &= \tilde{L} \\ T_k(\tilde{L}) &= 2\tilde{L}T_{k-1}(\tilde{L}) - T_{k-2}(\tilde{L}), \quad k = 2, 3, \ldots, K \end{aligned} \tag{12}$$

where $T_0(\tilde{L})$ represents the value of the zeroth-order Chebyshev polynomial, which is fixed at 1. $T_1(\tilde{L})$ denotes the value of the first-order Chebyshev polynomial, which is equal to the Laplacian matrix $\tilde{L}$. $T_k(\tilde{L})$ is the value of the kth order Chebyshev polynomial, calculated recursively based on the values of the previous two polynomial.

The feature propagation process of conventional Chebyshev graph convolution [40] can be described as

$$Y = \sum_{k=0}^{K} w_k T_k(\tilde{L}) X \tag{13}$$

where X is the input feature matrix containing feature vectors for the nodes in the graph. w_k is the coefficient for the kth-order Chebyshev polynomial and is a learnable parameter. Y represents the final output feature and is computed as the weighted sum of the Chebyshev polynomials combined with the input features X.

Unlike conventional Chebyshev graph convolution, we have reparameterized the coefficients of the Chebyshev polynomials. This reparameterization allows w_k not to be a simple learnable parameter; instead, it is weighted by the Chebyshev polynomials of the Chebyshev nodes x_n, establishing

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

a closer connection between w_k and x_n. The polynomial coefficients can be initialized based on values determined by the Chebyshev nodes x_n, and they are continuously updated during training according to the input features. This enables the model to dynamically adjust the shape of the filters in the training process [41]. The formulation for coefficient reparameterization is expressed as follows:

$$x_n = \cos\left(\frac{(K-n+0.5)\pi}{K+1}\right), \quad n = 0, 1, \ldots, K$$

$$w_k = \frac{2}{K+1}\sum_{n=0}^{K} \lambda_n \cdot T_k(x_n), \quad k = 0, 1, \ldots, K \tag{14}$$

where x_n represents the Chebyshev nodes, which are calculated during initialization for use in subsequent coefficient calculations. λ_n is the learnable parameter. The feature propagation process after reparameterization is expressed as

$$Y = \frac{2}{K+1}\sum_{k=0}^{K}\sum_{n=0}^{K} \lambda_n \cdot T_k(x_n) T_k(\tilde{L}) X. \tag{15}$$

This structure allows for dynamic and efficient feature propagation while leveraging structural properties of the graph through Chebyshev polynomial approximations, enhancing the model's ability to learn from the graph data.

E. Two-Branch Integration Module

The TIM is designed to integrate the outputs from BSEBM and RCGC branches to enhance the proposed model's classification capabilities by combining contextual information from both modalities.

As shown in Fig. 6, the outputs from BSEBM and RCGC branches are input into the TIM, where each output undergoes initial processing through pooling layers to reduce dimensionality and preserve essential spatial information. These pooling features are then passed through a series of 2-D convolutional layers. For each branch, the first convolutional layer captures local features, followed by an rectified linear unit (ReLU) activation function to introduce nonlinearity, and a second convolutional layer further refines the feature representation to ensure an informative output. The processed outputs from both branches are then fed into Sigmoid functions, which generate attention weights to highlight the importance of different feature channels. The output of each branch is multiplied by these attention weights, allowing the proposed model to focus on the most relevant features during the fusion process. The processed outputs from both BSEBM and RCGC branches are then combined through an elementwise addition operation. Finally, the fused feature representation is forwarded for classification tasks.

The pseudocode for the inference process of the DBMGNet is summarized in Algorithm 1.

F. Loss Function

The loss function of our DBMGNet is designed to optimize the performance of both individual branches and their fused output. The loss function comprises two types of losses: cross-entropy (CE) loss [42] and Kullback–Leibler (KL) [43]

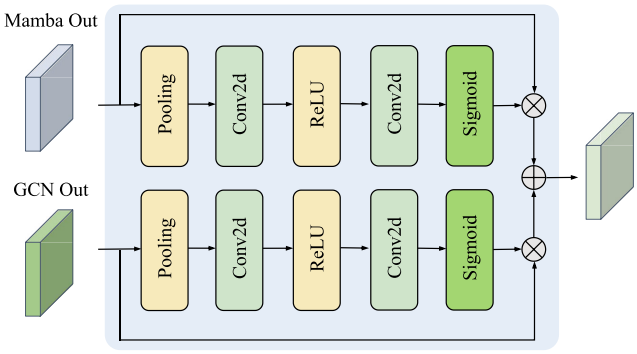


Fig. 6. TIM effectively integrates the outputs from BSEBM and RCGC branches.

Algorithm 1 Pseudo-Procedure of the Proposed DBMGNet

Input : Input patch data $X \in \mathbb{R}^{B\times C_{\text{in}}\times H\times W}$
Output: Predicted labels $Pred$.
1. Heterogeneous Spatial Convolution Block:
$X_{\text{emb}} = \text{Conv}_{1\times1}(X)$
$X_{\text{out}} = \text{Conv}_{1\times1}(\text{Conv}_{1\times1}(X_{\text{emb}}) + \text{Conv}_{3\times3}(X_{\text{emb}}))$
2. Band Selection Enhanced Bidirectional Mamba:
1) Spacial Bidirectional Mamba:
$X_{\text{c}} = \text{DSConv}(X_{\text{out}})$
$X_{\text{spaf}} = \text{Mamba}_{\text{spaf}}(X_c)$
$X_{\text{spab}} = \text{Flip}(\text{Mamba}_{\text{spab}}(\text{Flip}(X_c)))$
2) Band Selection Mamba:
$X_r = \text{Rearrange}(X_c)$
$X_b = \text{Mamba}_{\text{spef}}(X_r) + \text{Flip}(\text{Mamba}_{\text{speb}}(\text{Flip}(X_r)))$
$S = \text{Sigmoid}(\text{Linear}(\text{Pooling}(X_b)))$
3) Output of BSEBM:
$Y_{\text{spaf}} = X_{\text{spaf}} * S$
$Y_{\text{spab}} = X_{\text{spab}} * S$
$Y_{\text{BSEBM}} = \text{Conv}(\text{Concat}[\text{Conv}(Y_{\text{spaf}}), \text{Conv}(Y_{\text{spab}})])$
3. Reparameterized Chebyshev Graph Convolution:
1) Adjacency Matrix:
$S(i,j) = \frac{x_i \cdot x_j}{\|x_i\|\|x_j\|}$
$A(i,j) = \begin{cases} 1, & \text{if } S(i,j) > T \\ 0, & \text{if } S(i,j) \le T \end{cases}$
2) Feature Propagation:
$L = I - D^{-1/2} A D^{-1/2}$
$\tilde{L} = L + I$
$x_n = \cos\left(\frac{(K-n+0.5)\pi}{K+1}\right), \quad n = 0, 1, \ldots, K$
$Y = \frac{2}{K+1}\sum_{k=0}^{K}\sum_{n=0}^{K} \lambda_n \cdot T_k(x_n) T_k(\tilde{L}) X_{\text{out}}$
3) Output of RCGC:
$Y_{\text{RCGC}} = \text{MLP}(X_{\text{out}} + Y) + (X_{\text{out}} + Y)$
4. Two-Branch Integration Module:
$Y_{\text{TIM1}} = Y_{\text{BSEBM}} * \text{Sigmoid}(\text{Conv}(\text{ReLU}(\text{Conv}(\text{Pooling}(Y_{\text{BSEBM}})))))$
$Y_{\text{TIM2}} = Y_{\text{RCGC}} * \text{Sigmoid}(\text{Conv}(\text{ReLU}(\text{Conv}(\text{Pooling}(Y_{\text{RCGC}})))))$
$Y_{\text{Final}} = Y_{\text{TIM1}} + Y_{\text{TIM2}}$
5. Classifier:
$Pred = \text{Classifier}(\text{Pooling}(Y_{\text{Final}}))$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DETAILS OF EACH LAND-COVER CLASS OF FOUR DATASETS USED FOR EVALUATION IN THE EXPERIMENTS

Indian Pines (IP)				Pavia University (PU)				Houston 2013 (HU)				WHU-Hi-LongKou (LK)			
No.	Name	Train.	Test.	No.	Name	Train.	Test.	No.	Name	Train.	Test.	No.	Name	Train.	Test.
1	Corn Notill	50	1384	1	Asphalt	548	6304	1	Healthy Grass	198	1053	1	Corn	54	34457
2	Corn Mintill	50	784	2	Meadows	540	18146	2	Stressed Grass	190	1064	2	Cotton	53	8321
3	Corn	50	184	3	Gravel	392	1815	3	Synthetic Grass	192	505	3	Sesame	54	2977
4	Grass Pasture	50	447	4	Trees	524	2912	4	Tree	188	1056	4	Broad-Leaf Soybean	53	63159
5	Grass Trees	50	697	5	Painted metal Sheets	265	1113	5	Soil	186	1056	5	Narrow-Leaf Soybean	54	4097
6	Hay Windrowed	50	439	6	Bare Soil	532	4572	6	Water	182	143	6	Rice	50	11804
7	Soybean Notill	50	918	7	Bitumen	375	981	7	Residential	196	1072	7	Water	53	67003
8	Soybean Mintill	50	2418	8	Self-Blocking Bricks	514	3364	8	Commercial	191	1053	8	Roads and Houses	53	7071
9	Soybean Clean	50	564	9	Shadows	231	795	9	Road	191	1036	9	Mixed Weed	51	5178
10	Wheat	50	162	-	-	-	-	10	Highway	181	1054	-	-	-	-
11	Woods	50	1244	-	-	-	-	11	Railway	192	1041	-	-	-	-
12	Buildings Grass Trees Drives	50	330	-	-	-	-	12	Parking Lot1	184	285	-	-	-	
13	Stone Steel Towers	50	45	-	-	-	-	13	Parking Lot2	181	247	-	-	-	-
14	Alfalfa	15	39	-	-	-	-	14	Tennis Court	14	424	-	-	-	-
15	Grass Pasture Mowed	15	11	-	-	-	-	15	Running Track	187	473	-	-	-	-
16	Oats	15	5												
Total	-	695	9671	-	-	3921	40002	-	-	2832	12197	-	-	475	204067

divergence loss. The CE loss is defined as follows:

$$\mathcal{L}_{\mathrm{CE}}(y, \hat{y}) = -\sum_{i} y_i \log(\hat{y}_i) \tag{16}$$

where y represents the true distribution, and $\hat{y}$ is the probability distribution predicted by the model. Meanwhile, the KL divergence loss is employed to measure the difference between two probability distributions, and is defined as

$$\mathcal{L}_{\mathrm{KL}}(P \| Q) = \sum_{i} P(i) \log\left(\frac{P(i)}{Q(i)}\right) \tag{17}$$

where P is the true distribution, and Q is the predicted distribution.

Specifically, the CE loss term is calculated using the output of TIM and the individual outputs of BSEBM and RCGC branches. Based on (16), the three CE losses are shown as

$$\begin{aligned} \mathcal{L}_F &= \mathcal{L}_{\mathrm{CE}}(t, F) \\ \mathcal{L}_M &= \mathcal{L}_{\mathrm{CE}}(t, M) \\ \mathcal{L}_G &= \mathcal{L}_{\mathrm{CE}}(t, G) \end{aligned} \tag{18}$$

where F represents the final output of the DBMGNet. M is the output of a separate classification head that classifies the features from the BSEBM branch. G is the output of another separate classification head that classifies the features from the RCGC branch. t is the ground truth (GT).

To ensure the consistency between the decision results of the two branches, we incorporate the KL divergence loss in both directions. Based on (17), the KL loss term is expressed as

$$\begin{aligned} \mathcal{L}_{\mathrm{KL1}} &= \mathcal{L}_{\mathrm{KL}}(M \| G) \\ \mathcal{L}_{\mathrm{KL2}} &= \mathcal{L}_{\mathrm{KL}}(G \| M). \end{aligned} \tag{19}$$

The overall loss is a sum of these components

$$\mathcal{L} = \mathcal{L}_F + \mathcal{L}_M + \mathcal{L}_G + \mathcal{L}_{\mathrm{KL1}} + \mathcal{L}_{\mathrm{KL2}}. \tag{20}$$

By optimizing this loss function, our model aims to achieve accurate classification results while maintaining the consistency in decision outcomes by the BSEBM and RCGC branches.

III. EXPERIMENT

A. Data Description

To validate the performance of the proposed method, four widely used public HSI datasets are selected for extensive experiments, including Indian Pines (IP), Pavia University (PU), Houston (HU) 2013, and WHU-Hi-LongKou (LK). The fixed numbers of training and testing samples for each dataset are detailed in Table I.

1) Indian Pines: The IP dataset was acquired by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) Sensor over northwestern Indiana, USA. It comprises 220 spectral bands covering the range from 400 to 2500 nm, with an image size of 145 × 145 pixels and a ground sampling distance of 20 m. In the experiment, 20 water-absorption bands and noise bands were removed, and 200 bands were selected. There are 16 mainly investigated categories in this studied scene.

2) Pavia University: The PU dataset was acquired using the Reflective Optics System Imaging Spectrometer (ROSIS) sensor over PU and its surroundings in Italy. It comprises 103 spectral bands covering the range from 430 to 860 nm, with an image size of 610 × 340 pixels and a ground sampling distance of 1.3 m. There are nine mainly investigated categories in the studied scene.

3) HU 2013: The HU dataset was acquired by the ITRES CASI-1500 sensor over the campus of the University of HU and its neighboring rural areas, TX, USA. It comprises 144 spectral bands covering the range from 364 to 1046 nm, with an image size of 349 × 1905 pixels and a ground sampling distance of 10 m. There are 15 mainly investigated categories in the studied scene.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON THE IP DATASET

Class	CNN-based			Transformer-based			Dual-branch			GCN-based		Mamba-based		
	DFFN	A2S2K	SMESC	SF	SSFTT	Morphformer	DBMA	CTMixer	DBCTNet	FDGC	GraphGST	SSMamba	MIM	DBMGNet
	2018	2021	2024	2021	2022	2023	2019	2022	2024	2022	2024	2024	2025	Ours
1	79.41	92.56	67.49	48.41	85.04	86.92	78.47	94.51	94.08	91.04	**95.59**	80.27	92.56	94.65
2	95.03	97.96	81.51	88.01	93.49	97.07	92.60	98.85	99.11	90.56	94.64	93.37	98.85	**100.00**
3	97.28	**100.00**	88.04	90.22	98.37	99.46	98.91	**100.00**	**100.00**	100.00	98.91	95.65	98.91	**100.00**
4	96.20	96.20	90.38	94.63	95.97	97.76	92.62	97.54	95.97	95.53	94.63	95.97	96.42	**98.43**
5	98.13	**100.00**	91.68	86.23	99.28	96.27	98.42	99.71	**100.00**	89.67	98.71	**100.00**	**100.00**	99.86
6	98.86	**100.00**	98.18	90.89	99.54	99.54	99.54	**100.00**	**100.00**	**100.00**	99.54	99.77	99.54	**100.00**
7	90.09	93.14	77.34	81.81	86.82	97.39	92.59	94.55	90.74	96.41	95.75	93.14	95.21	**97.71**
8	88.30	91.15	72.04	69.40	74.52	95.04	76.22	86.85	92.14	83.17	87.34	82.05	88.54	**97.48**
9	89.18	92.20	71.63	64.18	90.07	88.83	91.67	96.81	93.97	83.16	95.21	92.73	94.33	**96.63**
10	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	98.15	99.38	**100.00**	99.38	**100.00**
11	95.34	98.39	96.86	89.95	93.97	93.17	95.18	99.12	98.07	**99.92**	95.82	96.30	98.63	99.20
12	95.15	**100.00**	86.97	86.36	97.88	99.54	98.48	**100.00**	**100.00**	90.91	97.27	95.45	99.70	**100.00**
13	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
14	94.87	**100.00**	94.87	92.31	**100.00**	97.44	92.31	**100.00**	**100.00**	94.87	**100.00**	76.92	**100.00**	**100.00**
15	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	90.91	**100.00**	**100.00**	**100.00**	**100.00**
16	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**	**100.00**
OA(%)	90.95	95.07	80.83	76.55	88.00	94.46	87.98	94.88	95.45	91.09	94.06	90.07	94.79	**98.00**
AA(%)	94.86	97.60	88.56	86.40	94.69	96.80	94.19	98.00	97.76	94.02	97.05	93.85	97.63	**99.00**
$\kappa\times100$	89.66	94.35	78.17	73.49	86.37	93.65	86.34	94.15	94.79	89.85	93.22	88.69	94.04	**97.71**

4) WHU-HI-LongKou: The LK dataset was acquired using the Headwall Nano-Hyperspec imaging sensor mounted on a DJI Matrice 600 Pro UAV platform over Longkou Town, Hubei Province, China. It comprises 270 spectral bands covering the range from 400 to 1000 nm, with an image size of 550 × 400 pixels and a ground sampling distance of approximately 0.463 m. There are nine mainly investigated categories in the studied scene.

B. Experimental Setups

1) Implementation Details: We implement our approach using the PyTorch framework and train it on a NVIDIA GeForce GTX 4090 GPU with 24 GB of memory. The minibatch size and epoch count are configured at 64 and 200, respectively, aiming to update training parameters across all frameworks. For a fair comparison, both the optimizer and learning rate scheduler are maintained at the default settings of the comparative methods. For our DBMGNet, the learning rate is set to 0.0001 and the optimizer is Adam, with a warm-up period of ten epochs at the beginning of training.

2) Evaluation Metrics: To quantitatively evaluate the performance of HSI classification, we utilize three metrics: overall accuracy (OA), average accuracy (AA), and kappa coefficient (κ). OA is defined as the percentage of samples correctly classified across all classes relative to the total number of samples. AA calculates the mean accuracy for each class. The κ coefficient quantifies the agreement between the GT and the classification outcomes.

3) Baseline Models: To demonstrate the effectiveness of DBMGNet, 13 representative methods are selected for comparative experiments: DFFN [9], A2S2K [44], SMESC [11], SF [23], SSFTT [24], Morphformer [25] DBMA [45], CTMixer [36], DBCTNet [37], FDGC [17], GraphGST [18], SSMamba [34], and MiM [35]. Among them, DFFN, A2S2K, and SMESC are CNN-based architectures, SF, SSFTT, and Morphformer are Transformer-based models, DBMA, CTMixer, and DBCTNet are dual-branch models, FDGC and GraphGST are graph-based models, and SSMamba and MiM are Mamba-based models.

C. Comparative Experiments

1) Quantitative Analysis: In the quantitative analysis, as shown in Tables II–V, we observe that our DBMGNet achieves the highest performance across four datasets in terms of OA, AA, and Kappa. For the IP dataset (Table II), our DBMGNet achieves an outstanding performance with an OA of 98.00%, an AA of 99.00%, and a Kappa of 97.71. Compared to DBCTNet, which achieves the highest OA (95.45%) and Kappa (94.79) among the compared methods, our method shows improvements of 2.55% in OA and 2.92 in Kappa. Similarly, compared to CTMixer, which attains the highest AA (98.00%), DBMGNet surpasses it by 1.00% in AA. Notably, DBMGNet achieves the highest classification accuracy in 13 out of the 16 categories of the IP dataset, demonstrating comprehensive improvements. In the PU dataset (Table III), our DBMGNet achieves an impressive OA of 96.68%, an AA of 97.01%, and a Kappa of 95.55. Compared to SSMamba, which achieves the highest OA (94.54%) and Kappa (92.60) among the compared methods, our method shows improvements of 2.14% in OA and 2.95 in Kappa. Similarly, compared to CTMixer, which attains the highest AA (92.92%), DBMGNet surpasses it by 4.09% in AA. For the HU dataset (Table IV), our DBMGNet achieves an OA of 91.98%, an AA of 93.11%, and a Kappa of 91.29. Compared to GraphGST, which achieves the highest OA (89.42%) and Kappa (88.51) among the compared methods, our method shows improvements of 2.56% in OA and 2.78 in Kappa. Similarly, compared to Morphformer, which attains the highest AA (90.92%), DBMGNet surpasses it by 2.19% in AA. In the LK dataset (Table V), our DBMGNet achieves an OA of 95.55%, an AA of 94.95%, and a Kappa of 94.22. Compared to CTMixer, which achieves the highest

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON THE PU DATASET

Class	CNN-based			Transformer-based			Dual-branch			GCN-based		Mamba-based		
	DFFN	A2S2K	SMESC	SF	SSFTT	Morphformer	DBMA	CTMixer	DBCTNet	FDGC	GraphGST	SSMamba	MIM	DBMGNet
	2018	2021	2024	2021	2022	2023	2019	2022	2024	2022	2024	2024	2025	Ours
1	79.19	92.47	87.36	71.86	86.47	86.88	92.21	92.45	89.93	66.50	89.02	91.86	88.17	**95.88**
2	95.60	72.97	98.27	75.85	59.47	83.26	83.44	94.70	71.40	92.71	75.12	**98.17**	71.17	96.22
3	**89.09**	72.23	53.61	69.31	78.79	85.01	76.25	87.77	87.55	86.89	80.44	69.53	79.06	86.89
4	95.60	76.03	**98.70**	95.81	79.26	83.99	88.12	97.22	79.70	83.31	92.48	97.80	83.10	98.39
5	98.56	99.46	98.65	99.37	99.46	99.82	**100.00**	99.10	99.10	**100.00**	99.55	99.73	**100.00**	99.73
6	89.61	99.04	93.61	68.15	98.67	97.00	97.73	86.42	**99.78**	72.13	94.16	87.80	96.94	98.91
7	84.30	99.08	87.97	66.67	94.70	97.66	92.86	99.69	**99.90**	87.46	98.06	90.21	96.94	98.27
8	98.84	98.75	96.43	92.95	92.69	98.69	98.90	90.49	97.65	64.60	91.80	98.72	93.61	**99.32**
9	95.72	98.11	96.60	93.84	99.12	97.36	98.87	88.43	99.12	72.70	99.12	96.98	97.23	**99.50**
OA(%)	92.11	83.26	93.59	77.72	76.08	87.93	88.77	93.03	83.13	82.59	84.12	94.54	81.86	**96.68**
AA(%)	91.84	89.79	90.13	81.53	87.62	92.19	92.04	92.92	91.57	80.70	91.08	92.31	89.58	**97.01**
$\kappa\times100$	89.42	78.56	91.35	71.04	70.23	84.21	85.31	90.64	78.51	76.85	79.55	92.60	76.89	**95.55**

TABLE IV
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON THE HU DATASET

Class	CNN-based			Transformer-based			Dual-branch			GCN-based		Mamba-based		
	DFFN	A2S2K	SMESC	SF	SSFTT	Morphformer	DBMA	CTMixer	DBCTNet	FDGC	GraphGST	SSMamba	MIM	DBMGNet
	2018	2021	2024	2021	2022	2023	2019	2022	2024	2022	2024	2024	2025	Ours
1	83.10	82.72	82.72	81.48	85.38	82.91	82.62	83.10	83.10	71.79	**90.22**	82.53	84.14	82.81
2	100.00	99.81	99.06	95.11	87.69	99.72	99.72	94.64	**99.91**	84.30	99.34	99.72	97.84	99.53
3	96.04	**100.00**	97.43	85.94	99.21	96.97	100.00	99.80	**100.00**	98.42	95.45	97.43	99.80	**100.00**
4	93.94	**100.00**	98.67	90.44	97.25	96.97	96.97	98.67	92.61	80.02	90.06	99.34	90.91	99.91
5	99.91	**100.00**	98.58	97.06	99.72	99.30	99.72	**100.00**	**100.00**	95.83	**100.00**	**100.00**	**100.00**	**100.00**
6	97.20	95.80	95.80	90.21	**100.00**	88.99	95.80	95.80	95.80	88.11	**100.00**	93.71	95.80	95.80
7	81.81	93.66	86.01	86.85	77.24	69.71	**91.88**	80.22	85.91	84.51	79.66	83.49	87.13	91.14
8	80.06	88.41	85.85	70.66	68.95	80.17	81.58	66.29	72.08	92.12	87.56	81.39	**93.64**	92.31
9	83.38	77.53	73.28	72.52	86.02	80.17	84.42	**86.40**	84.70	79.60	77.15	86.21	78.94	82.91
10	56.37	53.86	80.41	60.14	60.42	73.36	59.07	71.43	76.93	55.69	86.58	72.59	68.92	**79.34**
11	83.40	94.40	83.30	54.36	88.61	**97.53**	86.62	96.30	91.94	77.99	90.04	90.13	91.27	97.44
12	84.25	84.73	78.48	75.41	89.24	85.40	83.09	88.66	**90.49**	80.49	90.39	84.25	89.05	84.44
13	84.91	91.93	90.18	55.09	**98.25**	91.93	80.35	92.63	90.18	62.11	58.95	85.96	77.89	92.28
14	**100.00**	**100.00**	**100.00**	92.71	**100.00**	97.98	**100.00**	**100.00**	**100.00**	98.38	**100.00**	93.93	**100.00**	98.79
15	86.89	**100.00**	91.33	98.73	97.25	97.98	**100.00**	**100.00**	**100.00**	79.70	99.79	**100.00**	**100.00**	**100.00**
OA(%)	85.70	89.04	87.75	79.45	86.06	88.96	87.95	88.19	89.17	81.05	89.42	89.00	89.24	**91.98**
AA(%)	87.42	90.86	89.41	80.45	89.02	90.92	89.46	90.26	90.91	81.94	89.68	90.04	90.36	**93.11**
$\kappa\times100$	84.48	88.10	86.70	77.71	84.90	88.02	86.92	87.19	88.25	79.42	88.51	88.05	88.33	**91.29**

TABLE V
CLASSIFICATION RESULTS OF DIFFERENT METHODS ON THE LK DATASET

Class	CNN-based			Transformer-based			Dual-branch			GCN-based		Mamba-based		
	DFFN	A2S2K	SMESC	SF	SSFTT	Morphformer	DBMA	CTMixer	DBCTNet	FDGC	GraphGST	SSMamba	MIM	DBMGNet
	2018	2021	2024	2021	2022	2023	2019	2022	2024	2022	2024	2024	2025	Ours
1	93.35	99.17	96.62	94.40	96.31	96.62	91.70	99.27	95.34	94.81	97.77	98.44	96.99	**99.48**
2	81.04	96.62	88.11	70.29	91.84	88.11	92.30	95.01	93.35	97.88	91.21	93.62	96.30	**98.70**
3	82.43	84.28	84.21	76.35	93.11	84.21	79.61	86.33	98.05	**99.09**	91.00	83.24	98.66	86.50
4	78.95	86.29	83.68	80.24	74.66	83.68	78.56	86.12	87.84	77.11	86.03	86.21	78.17	**88.44**
5	48.28	95.63	94.19	73.13	95.48	94.19	94.70	**98.27**	93.73	95.02	97.61	93.99	87.55	96.53
6	94.91	99.05	98.25	91.45	93.03	98.25	94.92	98.41	91.98	91.00	97.47	98.17	99.29	**99.36**
7	99.75	99.63	99.94	**99.99**	98.22	99.94	99.36	99.94	99.77	93.88	99.98	99.81	99.75	99.95
8	89.89	95.62	94.97	79.51	74.57	94.97	90.95	92.32	**99.01**	89.92	95.33	95.12	89.49	95.22
9	75.09	86.60	91.15	81.09	79.18	91.15	86.58	89.19	83.18	72.21	80.24	87.04	83.97	**90.38**
OA(%)	88.94	94.49	93.03	89.15	88.61	93.03	90.09	94.49	94.02	88.25	93.95	94.18	91.42	**95.55**
AA(%)	82.63	93.65	92.35	82.94	88.49	92.35	89.85	93.87	93.58	90.10	92.96	92.85	92.24	**94.95**
$\kappa\times100$	85.80	92.86	91.00	86.03	85.43	91.00	87.30	92.87	92.23	85.08	92.16	92.46	88.99	**94.22**

OA (94.49%), AA (93.87%), and Kappa (92.87) among the compared methods, DBMGNet surpasses it by 1.06% in OA, 1.08% in AA, and 1.35 in Kappa. Overall, our DBMGNet consistently outperforms the CNN-based, Transformer-based, and dual-branch models across all datasets, demonstrating its superior classification accuracy and robustness. Additionally, in three out of the four datasets, DBMGNet achieves an OA improvement of over 2% compared to the best-performing methods, which is a considerable enhancement.

2) Visual Evaluation: Figs. 7–10 present a comparison of the classification result maps generated by the competing methods and our DBMGNet. The DBMGNet consistently

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

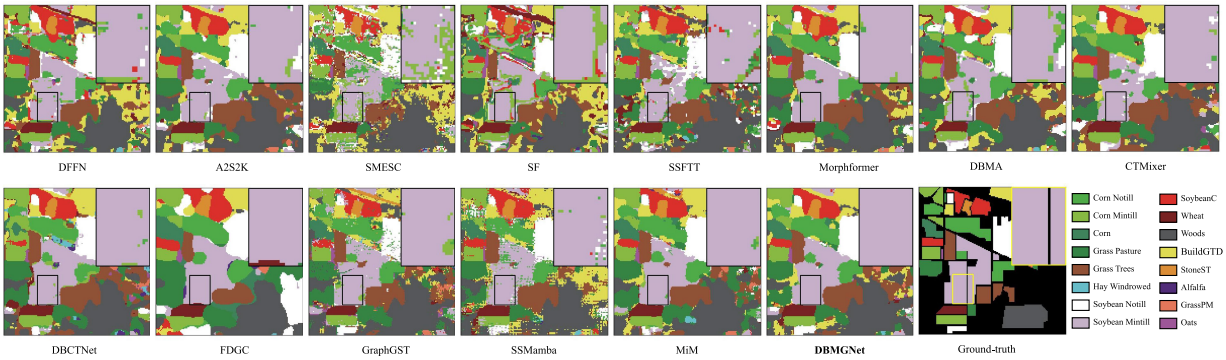


Fig. 7. Classification result maps on IP dataset. The zoomed-in view region for the Soybean-MinTill class shows that our DBMGNet is closest to the GT and exhibits minimal noise compared to other methods.

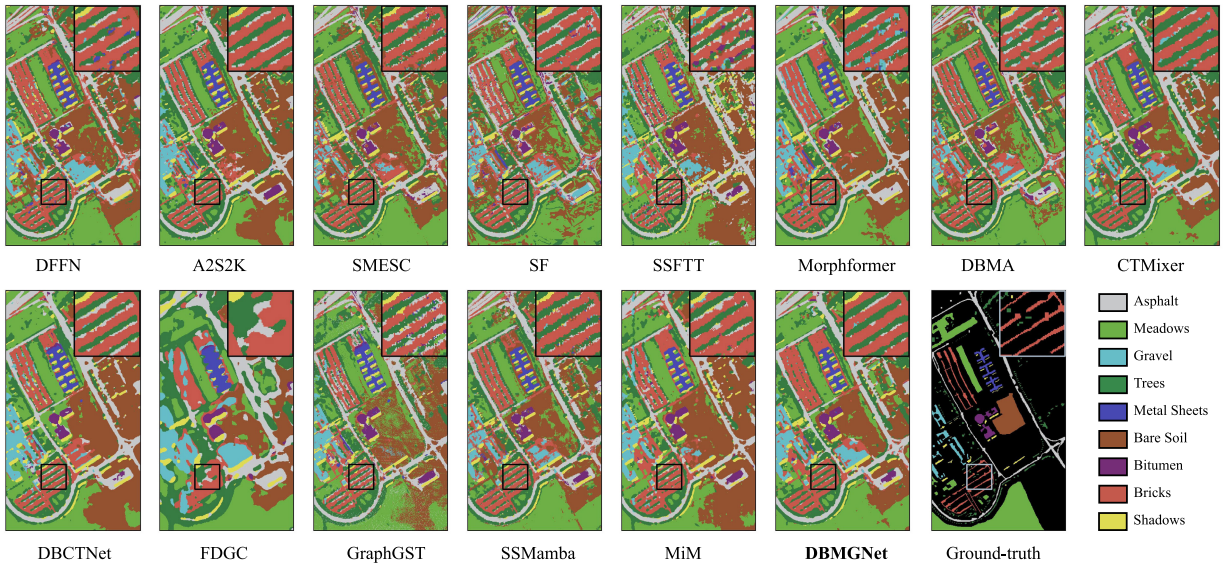


Fig. 8. Classification result maps on PU dataset. The zoomed-in view region for the Bricks class shows that our DBMGNet is closest to the GT and exhibits minimal noise compared to other methods.

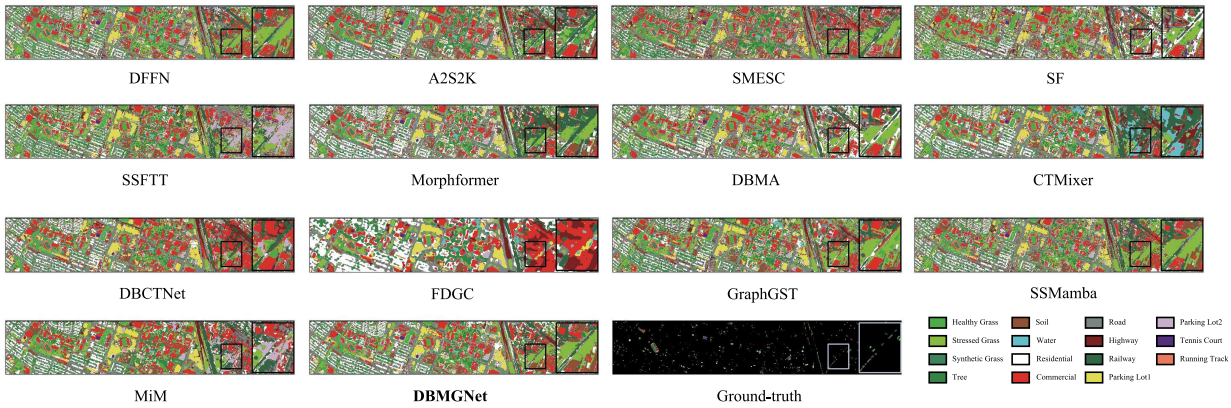


Fig. 9. Classification result maps on the HU dataset. The zoomed-in view region for the Railway class shows that our DBMGNet is closest to the GT and exhibits minimal noise compared to other methods.

produces high-quality classification maps, exhibiting fewer misclassifications and less noise than other approaches. For the IP dataset, in the zoomed-in view region of Fig. 7 focusing on the Soybean-MinTill class, our DBMGNet achieves classification results that are closest to the GT and exhibit minimal noise compared to other methods. In the PU dataset, the zoomed-in view region of Fig. 8 for the Bricks class shows that our DBMGNet achieves more precise classification. For the HU dataset, DBMGNet provides a clearer classification of the Railway class, as shown in Fig. 9. In the LK dataset, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

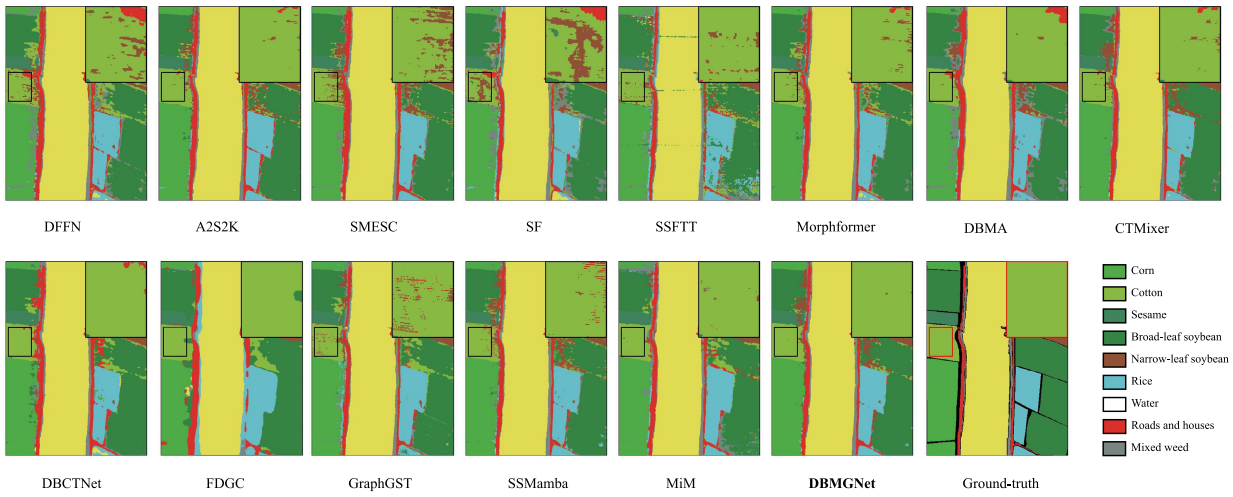


Fig. 10. Classification result maps on LK dataset. The zoomed-in view region for the Cotton class shows that our DBMGNet is closest to the GT and exhibits minimal noise compared to other methods.

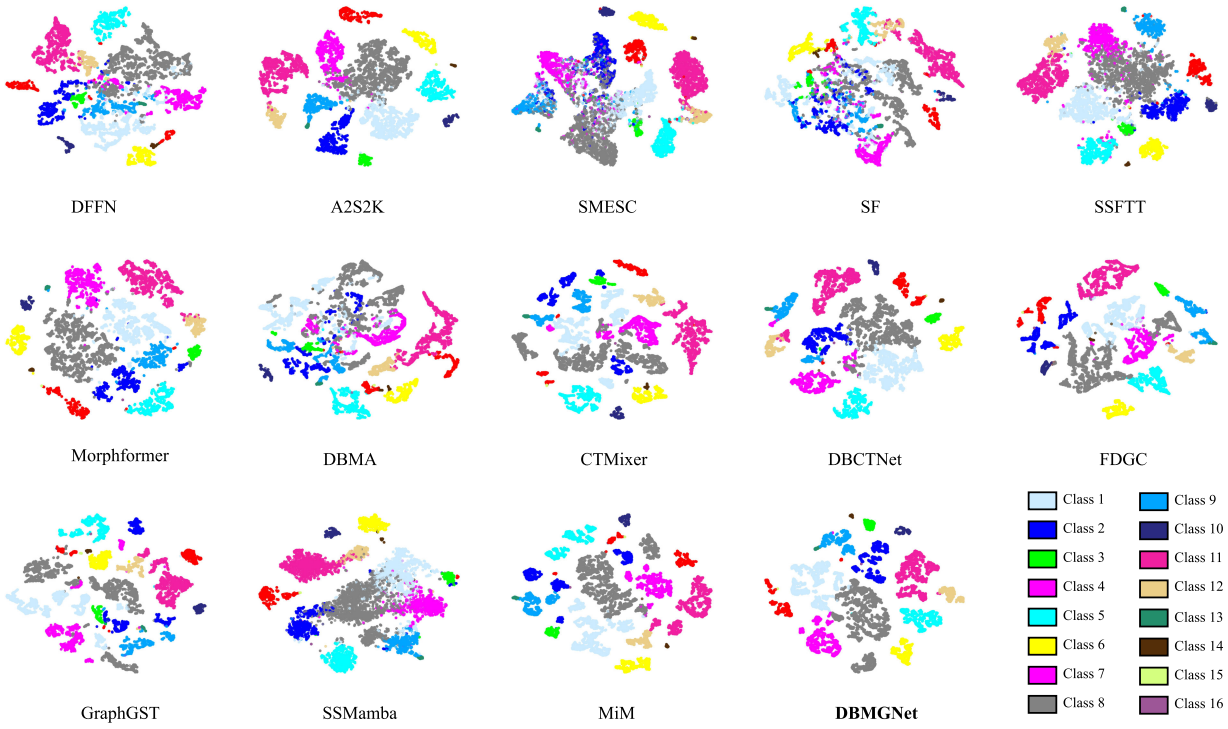


Fig. 11. t-SNE visualization on the IP dataset.

zoomed-in view region of Fig. 10 for the Cotton class reveals that DBMGNet achieves classification results with virtually no misclassification. Overall, the results obtained with DBMGNet are closer to the GT.

Moreover, Figs. 11–14 present t-SNE [46] visualizations for comparing classification features of each method. The results clearly show that our DBMGNet produces more distinct clustering, with tighter intraclass grouping and better separation between interclass features. These outcomes highlight the superior classification performance of our DBMGNet.

D. Ablation Study

BSEBM, RCGC, and TIM are the three main components of our model. To demonstrate their effectiveness, we conducted ablation experiments on the four datasets. As shown in Table VI, the full model consistently achieves the highest performance in terms of OA, AA, and Kappa. Removing either the BSEBM or RCGC branch results in a significant decline in model performance, demonstrating the necessity of the dual-branch approach. Additionally, when the TIM is removed and a

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

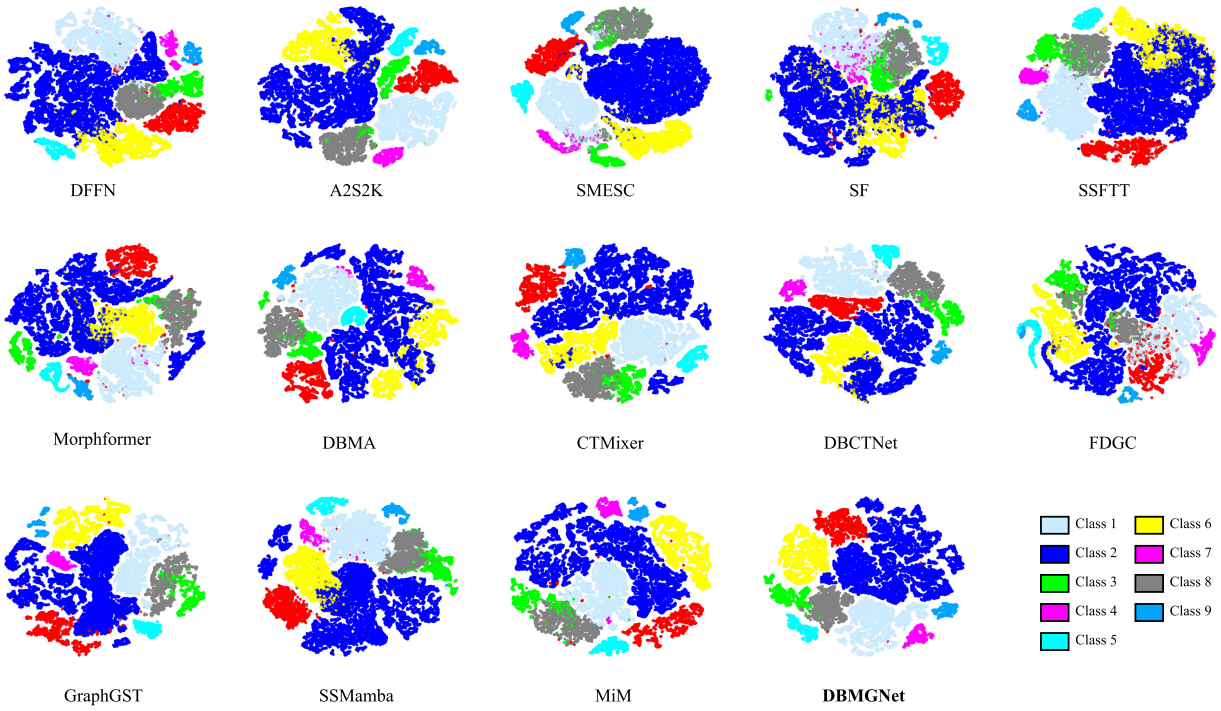


Fig. 12. t-SNE visualization on the PU dataset.

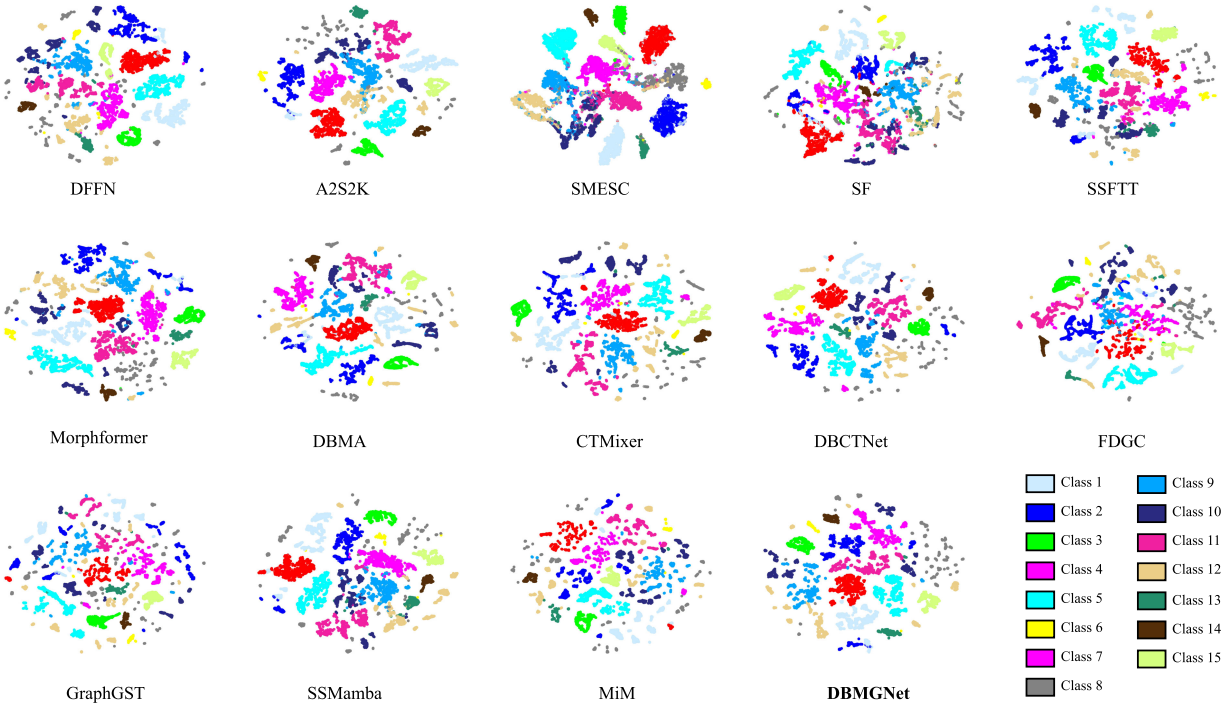


Fig. 13. t-SNE visualization on the HU dataset.

simple addition of the dual-branch features is used as the final feature representation, the model's performance also degrades, which confirms the effectiveness of the TIM. The results validate the effectiveness of integrating BSEBM, RCGC, and TIM in achieving superior classification performance.

In addition, we conducted an analysis to evaluate the effectiveness of our designed adjacency matrix. We compared our approach with two alternative adjacency matrix construction methods: one based on K-nearest neighbors (KNNs) [47] and the other based on prior edges [48]. As shown in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

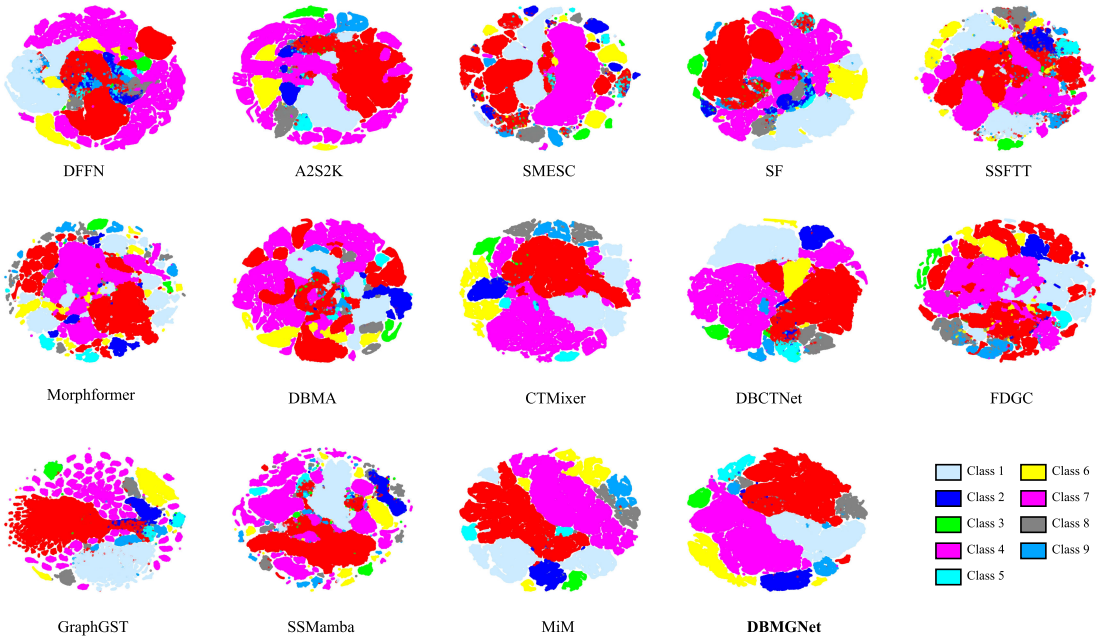


Fig. 14. t-SNE visualization on the LK dataset.

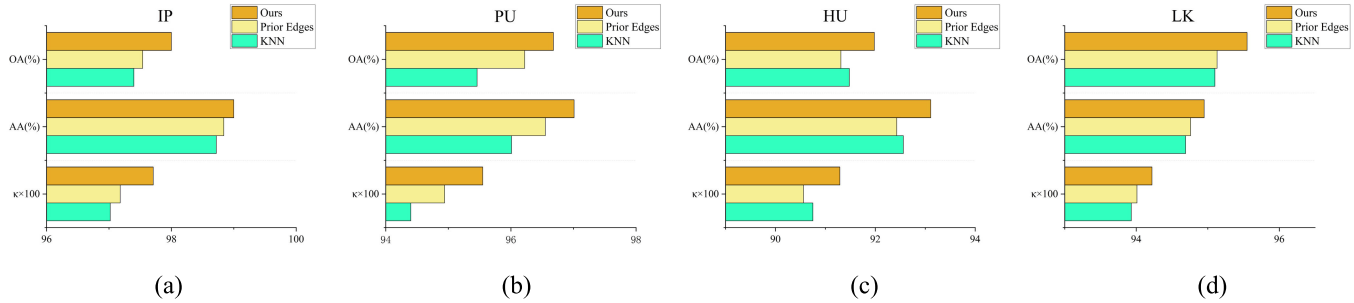


Fig. 15. Effectiveness of the adjacency matrix. (a) IP. (b) PU. (c) HU. (d) LK.

Fig. 15, the experimental results demonstrate that our designed adjacency matrix outperforms both alternatives, validating its effectiveness. We also analyze the role of BSM in the BSEBM branch by conducting experiments in two settings: first, using only the BSEBM branch (without the RCGC branch), and second, by further removing BSM from the BSEBM branch. The experimental results, as shown in Fig. 16, demonstrate that BSM helps the BSEBM branch extract more diverse features.

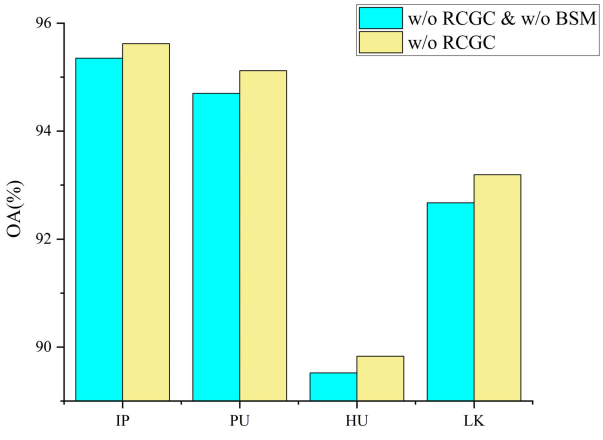


Fig. 16. Effectiveness of the BSM.

E. Effects of the Loss Function

Our loss function consists of three CE losses and two KL divergence losses. The three CE losses are computed based on the final output of the model and the outputs from each of the dual branches. To demonstrate the effectiveness of our designed loss function, we compare it with two alternative configurations: using a single CE loss based solely on the final output of the model, and using a three-CE-loss setup that includes the final output and the outputs from each of the dual branches. As shown in Fig. 17, the comparison scores exhibit a similar trend across all four datasets. The three-CE-loss configuration performs better than the single

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

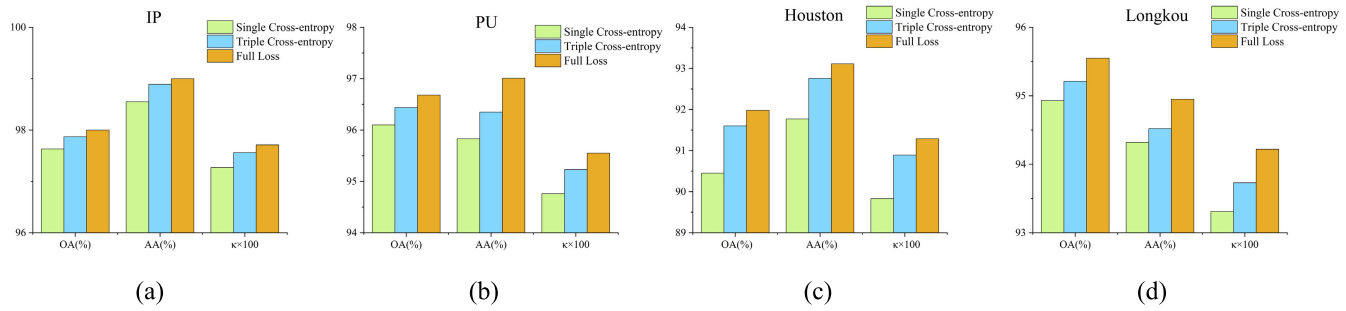


Fig. 17. Effects of the loss function. (a) IP. (b) PU. (c) HU. (d) LK.

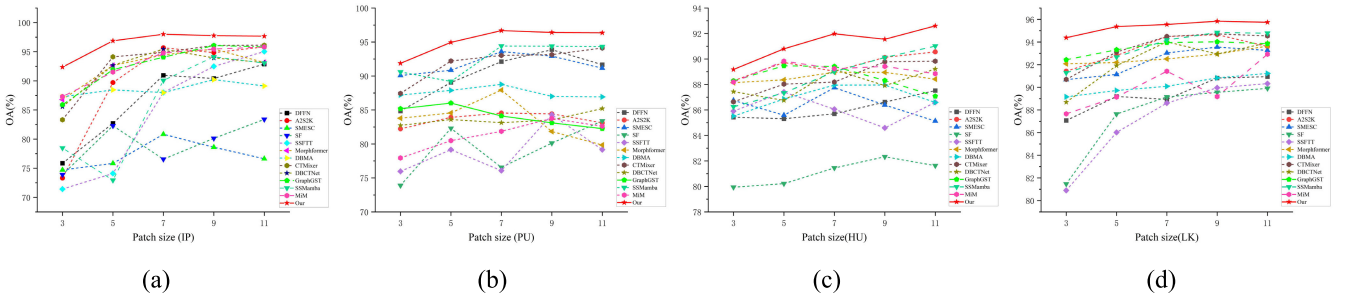


Fig. 18. Analysis of image patch size. (a) IP. (b) PU. (c) HU. (d) LK.

TABLE VI
ABLATION STUDY OF VARIOUS MODEL CONFIGURATIONS ACROSS FOUR DATASETS

Dataset	Configuration	OA(%)	AA(%)	$\kappa\times100$
IP	Without BSEBM	96.84	98.12	96.33
	Without RCGC	95.62	98.14	95.03
	Without TIM	97.13	98.32	96.82
	Full model	**98.00**	**99.00**	**97.71**
PU	Without BSEBM	96.03	96.16	94.62
	Without RCGC	95.12	96.01	93.47
	Without TIM	96.13	96.43	95.02
	Full model	**96.68**	**97.01**	**95.55**
HU	Without BSEBM	90.38	92.02	90.26
	Without RCGC	89.83	91.51	88.97
	Without TIM	91.32	92.42	90.56
	Full model	**91.98**	**93.11**	**91.29**
LK	Without BSEBM	94.02	93.71	92.54
	Without RCGC	93.19	93.05	91.21
	Without TIM	94.87	94.52	93.65
	Full model	**95.55**	**94.95**	**94.22**

CE loss, demonstrating the effectiveness of adding CE losses computed from the outputs of each branch. Furthermore, the addition of KL divergence losses further improves the model's performance compared to the three-CE-loss setup, validating the effectiveness of incorporating KL divergence losses. These results indicate that our designed loss function enables the model to achieve superior performance.

F. Analysis of Image Patch Size

We analyze the stability of our model under different patch sizes. Patch size plays a crucial role in determining the amount of spatial context available for feature extraction, which can significantly influence the model's ability to capture relevant information from an image. Generally, larger patch sizes provide more classification information, enabling the model to achieve more accurate classification results. We conduct comparative experiments on the four datasets using five different patch sizes: 3, 5, 7, 9, and 11. As shown in the results in Fig. 18, our DBMGNet consistently outperforms the competing models across all patch size settings. This demonstrates the stability of our DBMGNet under varying patch sizes.

G. Analysis of Model Computational Cost

To analyze the model computational cost, we compare DBMGNet with 13 baseline methods in terms of training time, testing time, the number of parameters, and computational complexity, as shown in Table VII. The experimental results demonstrate that our DBMGNet has shorter training and testing times than the Mamba-based SSMamba and MiM models, proving its superior runtime efficiency. In addition, we find that setting the model's dimension to 256 achieves good performance, but it also results in a slightly higher number of parameters. We reduce the model's dimension to 128 to explore a more lightweight version, and name this configuration as DBMGNet-Tiny (DBMGNet-T). The parameter count and computational complexity of DBMGNet-T are significantly reduced; meanwhile, DBMGNet-T still outperforms all baseline methods across four datasets. The four metrics of DBMGNet-T are all lower than those of the classic Transformer-based SF and are at a relatively low level compared to other methods. DBMGNet-T provides

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPUTATIONAL COST OF THE PROPOSED DBMGNET AND COMPARISON METHODS

Dataset	Metric	DFFN	A2S2K	SMESC	SF	SSFTT	Morphformer	DBMA	CTMixer	DBCTNet	FDGC	GraphGST	SSMamba	MiM	DBMGNet-T	DBMGNet
IP	Train time (s)	128.78	222.67	85.69	158.16	27.19	79.25	137.65	50.68	68.84	63.87	55.14	93.37	692.77	63.71	69.84
	Test time (s)	2.18	1.98	3.14	3.61	0.48	3.02	1.61	1.53	1.26	1.89	1.14	2.11	19.98	1.78	1.91
	Params (M)	0.52	0.371	0.564	1.041	0.153	0.278	0.606	0.617	0.030	2.384	2.355	0.354	1.171	0.434	1.454
	FLOPs (G)	0.025	0.102	0.021	0.158	0.001	0.015	0.146	0.030	0.008	0.015	0.002	0.019	0.051	0.017	0.059
PU	Train time (s)	666.05	469.64	460.49	774.62	105.14	436.86	240.18	236.08	183.09	303.84	165.23	497.75	3604.65	365.64	374.75
	Test time (s)	21.14	9.26	16.30	31.44	5.09	22.45	9.02	10.94	7.25	20.28	11.55	21.16	192.42	15.75	18.10
	Params (M)	0.506	0.221	0.152	0.442	0.153	0.227	0.321	0.650	0.016	1.922	1.867	0.333	1.171	0.394	1.374
	FLOPs (G)	0.025	0.052	0.006	0.053	0.001	0.010	0.087	0.032	0.004	0.014	0.001	0.011	0.051	0.016	0.055
HU	Train time (s)	433.90	364.31	308.15	506.99	68.39	272.58	176.63	41.99	163.00	206.96	240.76	359.65	2372.02	243.41	255.71
	Test time (s)	66.68	30.11	64.43	105.50	17.47	75.16	40.24	38.53	23.00	61.46	72.99	71.42	615.38	54.70	59.99
	Params (M)	0.512	0.283	0.298	0.649	0.153	0.233	0.443	1.371	0.022	2.318	6.989	0.341	1.171	0.412	1.410
	FLOPs (G)	0.025	0.073	0.011	0.091	0.001	0.011	0.104	0.067	0.005	0.015	0.003	0.014	0.051	0.016	0.057
LK	Train time (s)	511.16	336.83	467.71	721.36	101.16	407.42	356.07	267.08	228.75	289.97	55.14	437.72	3699.66	355.30	385.51
	Test time (s)	27.78	18.82	24.63	48.64	5.45	28.66	23.33	13.96	10.17	24.69	16.82	27.98	203.51	19.04	19.83
	Params (M)	0.530	0.480	1.015	1.706	0.153	0.333	0.809	0.208	0.040	1.922	7.447	0.375	1.171	0.458	1.502
	FLOPs (G)	0.026	0.138	0.038	0.266	0.001	0.020	0.199	0.010	0.011	0.014	0.005	0.025	0.051	0.019	0.062

a good choice for scenarios with low computational cost requirements.

IV. CONCLUSION

This article introduces a dual-branch hybrid network that combines Mamba and GCN for HSI classification. By leveraging Mamba's capacity for long-range dependence modeling, together with graph convolution's strength in capturing sample similarity, our approach significantly enhances classification accuracy. In the BSEBM branch, we leverage Mamba to simultaneously process spectral and spatial information, integrating both for better feature representation. Meanwhile, in the RCGC branch, we are the first to apply RCGC to HSI classification, and based on the intrinsic characteristics of hyperspectral data, we design an effective adjacency matrix to better define the graph structure. Additionally, we introduce the TIM that effectively fuses features from both branches. Numerous experimental results demonstrate that the proposed DBMGNet achieves state-of-the-art performance against other 13 approaches, excelling in both qualitative and quantitative comparisons. Despite these advantages, our DBMGNet still faces the issue of class accuracy imbalance, with lower classification accuracy on certain challenging categories (e.g., the Highway class in the HU dataset). In the future, we will explore solutions to address class accuracy imbalance and improve model's classification capability in difficult scenarios. Furthermore, we will explore the potential of Mamba hybrid architectures and apply them to other vision fields.

REFERENCES

[1] P. Duan, X. Kang, P. Ghamisi, and S. Li, "Hyperspectral remote sensing benchmark database for oil spill detection with an isolation forest-guided unsupervised detector," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5509711.

[2] P. Duan, T. Shan, X. Kang, and S. Li, "Spectral super-resolution in frequency domain," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Oct. 29, 2024, doi: 10.1109/TNNLS.2024.3481060.

[3] P. Duan, Z. Xie, X. Kang, and S. Li, "Self-supervised learning-based oil spill detection of hyperspectral images," *Sci. China Technolog. Sci.*, vol. 65, no. 4, pp. 793–801, Apr. 2022.

[4] X. Cao, F. Zhou, L. Xu, D. Meng, Z. Xu, and J. Paisley, "Hyperspectral image classification with Markov random fields and a convolutional neural network," *IEEE Trans. Image Process.*, vol. 27, no. 5, pp. 2354–2367, May 2018.

[5] M. Pal and G. M. Foody, "Feature selection for classification of hyperspectral data by SVM," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 5, pp. 2297–2307, May 2010.

[6] L. Velásquez, J. P. Cruz-Tirado, R. Siche, and R. Quevedo, "An application based on the decision tree to classify the marbling of beef by hyperspectral imaging," *Meat Sci.*, vol. 133, pp. 43–50, Nov. 2017.

[7] P. Duan, P. Ghamisi, X. Kang, B. Rasti, S. Li, and R. Gloaguen, "Fusion of dual spatial information for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 8, pp. 7726–7738, Sep. 2020.

[8] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[9] W. Song, S. Li, L. Fang, and T. Lu, "Hyperspectral image classification with deep feature fusion network," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 6, pp. 3173–3184, Jun. 2018.

[10] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[11] C. Yu, Y. Zhu, M. Song, Y. Wang, and Q. Zhang, "Unseen feature extraction: Spatial mapping expansion with spectral compression network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5521915.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.

[13] W. Liu, B. Liu, P. He, Q. Hu, K. Gao, and H. Li, "Masked graph convolutional network for small sample classification of hyperspectral images," *Remote Sens.*, vol. 15, no. 7, p. 1869, Mar. 2023.

[14] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, Jan. 2018, vol. 32, no. 1, pp. 1–8.

[15] S. Chen, K. Xu, X. Jiang, and T. Sun, "Spatiotemporal-spectral graph convolutional networks for skeleton-based action recognition," in *Proc. IEEE Int. Conf. Multimedia Expo. Workshops (ICMEW)*, Jul. 2021, pp. 1–6.

[16] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2020.

[17] Q. Liu, Y. Dong, Y. Zhang, and H. Luo, "A fast dynamic graph convolutional network and CNN parallel network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5530215.

[18] M. Jiang et al., "GraphGST: Graph generative structure-aware transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504016.

[19] J. Bai et al., "Hyperspectral image classification based on superpixel feature subdivision and adaptive graph structure," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5524415.

[20] Y. Su et al., "ACGT-net: Adaptive cuckoo refinement-based graph transfer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5521314.

[21] Y. Ding, Y. Chong, S. Pan, and C. Zheng, "Diversity-connected graph convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5518118.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.

[22] L. Yu et al., "DSPG: A new discriminant superpixel graph regularizer and convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5526118.
[23] D. Hong et al., "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5518615.
[24] L. Sun, G. Zhao, Y. Zheng, and Z. Wu, "Spectral–spatial feature tokenization transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, Jan. 2022, Art. no. 5522214.
[25] S. K. Roy, A. Deria, C. Shah, J. M. Haut, Q. Du, and A. Plaza, "Spectral–spatial morphological attention transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5503615.
[26] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2021, pp. 1–27.
[27] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.
[28] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision mamba: Efficient visual representation learning with bidirectional state space model," 2024, *arXiv:2401.09417*.
[29] Y. Liu et al., "VMamba: Visual state space model," 2024, *arXiv:2401.10166*.
[30] Z. Xing, T. Ye, Y. Yang, G. Liu, and L. Zhu, "SegMamba: Long-range sequential modeling mamba for 3D medical image segmentation," 2024, *arXiv:2401.13560*.
[31] K. Li et al., "VideoMamba: State space model for efficient video understanding," 2024, *arXiv:2403.06977*.
[32] K. Chen, B. Chen, C. Liu, W. Li, Z. Zou, and Z. Shi, "RSMamba: Remote sensing image classification with state space model," 2024, *arXiv:2403.19654*.
[33] D. Liang et al., "PointMamba: A simple state space model for point cloud analysis," 2024, *arXiv:2402.10739*.
[34] L. Huang, Y. Chen, and X. He, "Spectral–spatial mamba for hyperspectral image classification," *Remote Sens.*, vol. 16, no. 13, p. 2449, Jul. 2024.
[35] W. Zhou, S.-I. Kamata, H. Wang, M. S. Wong, and H. Hou, "Mamba-in-mamba: Centralized mamba-cross-scan in tokenized mamba model for hyperspectral image classification," *Neurocomputing*, vol. 613, Jan. 2025, Art. no. 128751.
[36] J. Zhang, Z. Meng, F. Zhao, H. Liu, and Z. Chang, "Convolution transformer mixer for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.
[37] R. Xu, X.-M. Dong, W. Li, J. Peng, W. Sun, and Y. Xu, "DBCT-Net: Double branch convolution-transformer network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5509915.
[38] X. Liu, A. H.-M. Ng, L. Ge, F. Lei, and X. Liao, "Multibranch fusion: A multibranch attention framework by combining graph convolutional network and CNN for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5528817.
[39] Y. Lu, X. Wang, S. Mei, F. Xu, and M. Ma, "DGT: Deformable graph transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5534815.
[40] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2016, pp. 1–9.
[41] M. He, Z. Wei, and J.-R. Wen, "Convolutional neural networks on graphs with Chebyshev approximation, revisited," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2022, pp. 7264–7276.
[42] A. Mao, M. Mohri, and Y. Zhong, "Cross-entropy loss functions: Theoretical analysis and applications," in *Proc. Int. Conf. Mach. Learning (ICML)*, Jan. 2023, pp. 23803–23828.
[43] S. Kullback and R. A. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, vol. 22, no. 1, pp. 79–86, 1951.
[44] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2021.
[45] W. Ma, Q. Yang, Y. Wu, W. Zhao, and X. Zhang, "Double-branch multi-attention mechanism network for hyperspectral image classification," *Remote Sens.*, vol. 11, no. 11, p. 1307, Jun. 2019.
[46] L. Van der Maaten and G. E. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 86, pp. 2579–2605, Jan. 2008.
[47] H. Wang, "Spectral graph reasoning network for hyperspectral image classification," in *Proc. 29th Int. Conf. Artif. Neural Netw. Artif. Neural Netw. Mach. Learning (ICANN)*, Bratislava, Slovakia. Cham, Switzerland: Springer, Sep. 2020, pp. 711–723.
[48] F. Zhao, J. Zhang, Z. Meng, H. Liu, Z. Chang, and J. Fan, "Multiple vision architectures-based hybrid network for hyperspectral image classification," *Expert Syst. Appl.*, vol. 234, Dec. 2023, Art. no. 121032.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 21:00:42 UTC from IEEE Xplore. Restrictions apply.